Movie Discovery, Review & Cinema Booking Database

Database Design Document

Name	ID No.	Name	ID No.	Name	ID No.
Sashwath Swaminathan	2024A3PS0361H	Vinny Polamarasetty	2024A1PS0930H	Srujana N S	2024A1PS0874H
Anushka Sahoo	2024A1PS0937H	Dhwaj Kishore	2024A1PS1187H		

Tech: Oracle 21c XE, SQL / PL/SQL, SQL Developer

1. What our project is about

Right now if you're into movies you basically use two different apps: something like Letterboxd to rate and review films, and something like BookMyShow to actually book tickets. So the info about what a person likes and what they actually go and watch never ends up in the same place. Our idea was to design one database that handles both halves: the **discovery side** (movies, genres, cast, ratings, reviews, watchlists) and the **cinema side** (theatres, screens, seats, shows, bookings, payments).

A big goal for us was to let the database itself enforce the important rules (like "one seat can't be sold twice for the same show") using constraints wherever possible, and only use a trigger when a constraint just can't do it. On top of this we wrote PL/SQL procedures (Section 8) and a web app that calls them; the app never re-checks a rule, it just shows Oracle's error.

Two small naming changes from the suggested table list: **USER** became **APP_USER** because USER is a reserved word in Oracle, and **SHOW** became **MOVIE_SHOW** because SHOW is a SQL*Plus command and it was messing up our scripts.

2. Assumptions

- All theatres are in India, prices are in rupees and times are local. We use Oracle DATE, which stores both date and time.
- One show = one movie on one screen at one start time. Different languages/formats of a movie are treated as separate movies.
- A seat's type (REGULAR or PREMIUM) is fixed for the screen, it doesn't change per show. Premium = base price + Rs 100.
- A booking is for exactly one show. You can't mix seats from two different shows in one booking.
- Each booking has at most one payment record. If a payment is retried we just update its status.
- Users can review any movie, even if they didn't book it through our system (same as Letterboxd).
- When a booking is cancelled the seats are released, but the booking row stays (status CANCELLED) so we keep the history.
- No login/passwords, real payment gateway or seat-hold timers. Payment is simulated, and in the app you pick which sample user you are acting as.

3. Entities at a glance

Entity	What it stores	Type
APP_USER	people who rate, review, keep watchlists and book tickets	strong
MOVIE	title, release date, duration, language, certificate (U/UA/A/S)	strong
GENRE, ACTOR	kept in their own tables so names aren't repeated for every movie	strong
THEATRE	a physical cinema (name, city, address)	strong
SCREEN	an auditorium inside a theatre (2D / 3D / IMAX)	weak (owner: THEATRE)
SEAT	a physical seat in a screen, e.g. row B seat 4; REGULAR or PREMIUM	weak (owner: SCREEN)
MOVIE_SHOW	one screening: a movie on a screen at a date/time, with a base price	strong
BOOKING	one purchase by one user for one show (PENDING / CONFIRMED / CANCELLED)	strong
PAYMENT	how and when a booking was paid (UPI / CARD / NETBANKING / WALLET)	strong, 1:1 with BOOKING
REVIEW, WATCHLIST, MOVIE_GENRE, MOVIE_ACTOR, BOOKING_SEAT	these come from the M:N relationships (see Section 5)	associative

4. ER / EER Diagram

We drew the diagram in Chen notation. To keep it readable, only the key attribute is shown inside each entity; all the other attributes are listed in the relational schema in Section 5.

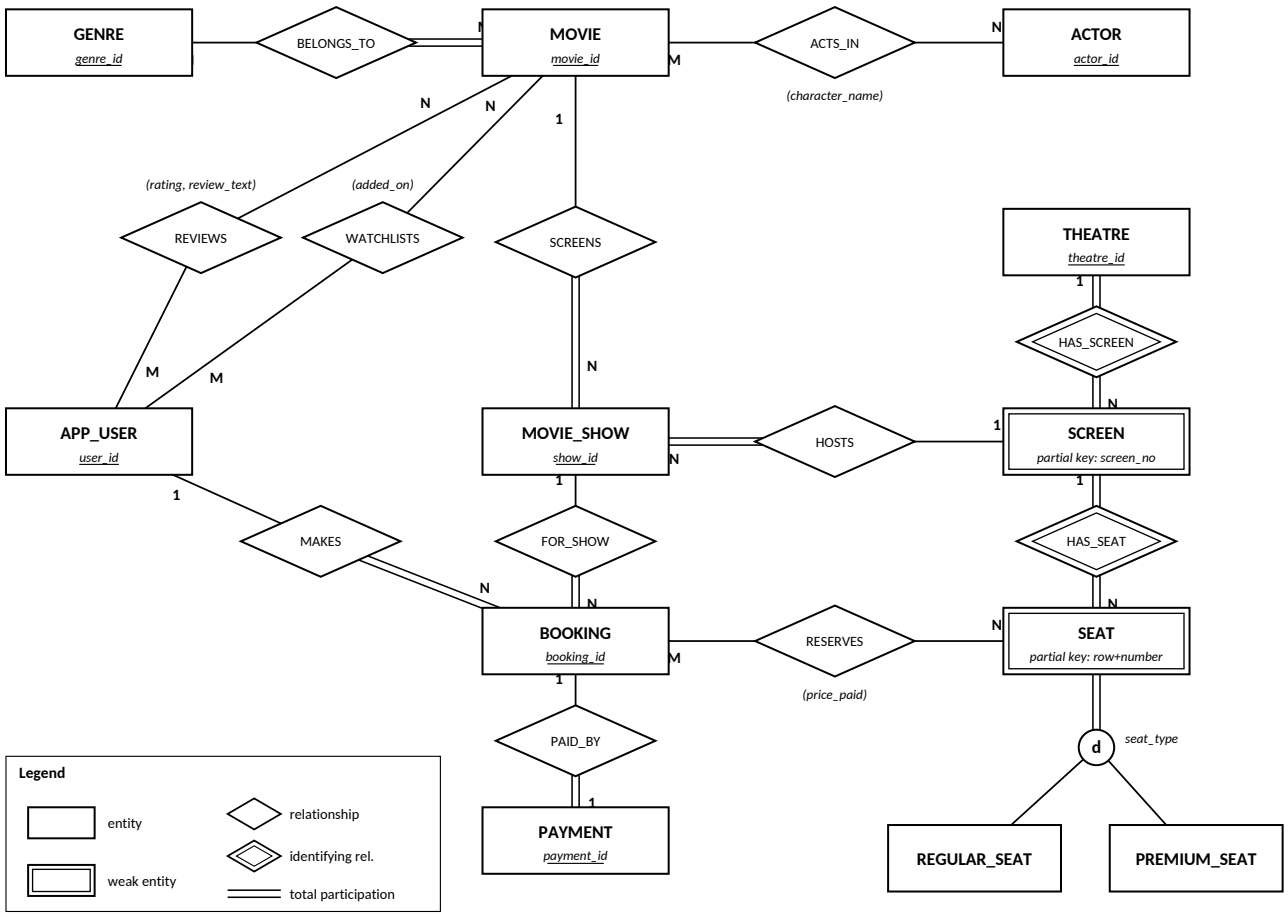


Figure 1: ER/EER diagram (double box = weak entity, double diamond = identifying relationship, double line = total participation, d = disjoint specialization)

Some things worth explaining about the diagram:

- **SCREEN and SEAT are weak entities.** "Screen 2" or seat "B4" means nothing unless you know which theatre/screen it's in, so they depend on their owner through identifying relationships (HAS_SCREEN, HAS_SEAT).
- **SEAT specialization (EER):** SEAT is split into REGULAR_SEAT and PREMIUM_SEAT based on seat_type. It's disjoint (a seat is exactly one type) and total (every seat is one of them).
- **The M:N relationships** (BELONGS_TO, ACTS_IN, REVIEWS, WATCHLISTS, RESERVES) become their own tables. Some of them have their own attributes, e.g. REVIEWS has rating and review_text and RESERVES has price_paid.
- **BOOKING is a proper entity, not just a link between user and show.** The same user can book the same show twice (two separate purchases), so (user_id, show_id) can't be the key.
- **PAID_BY is 1:1.** We kept PAYMENT separate from BOOKING because a booking exists before it's paid, and the payment has its own attributes and lifecycle (PENDING, SUCCESS, FAILED, REFUNDED).

5. Relational Schema (with primary and foreign keys)

Notation: underlined = primary key, *italic* = foreign key. We ended up with 15 tables in total.

Table	Attributes	Foreign keys / other keys
APP_USER	user_id, username, full_name, email, phone, city, date_of_birth, joined_on	UNIQUE username, UNIQUE email, UNIQUE phone
MOVIE	movie_id, title, release_date, duration_min, movie_language, certificate, description	UNIQUE (title, release_date)
GENRE	genre_id, genre_name	UNIQUE genre_name
ACTOR	actor_id, actor_name, nationality	-

Table	Attributes	Foreign keys / other keys
MOVIE_GENRE	*movie_id, genre_id*	movie_id → MOVIE, genre_id → GENRE (on delete cascade)
MOVIE_ACTOR	*movie_id, actor_id*, character_name	movie_id → MOVIE, actor_id → ACTOR (cascade)
REVIEW	*user_id, movie_id*, rating, review_text, review_date	user_id → APP_USER, movie_id → MOVIE (cascade)
WATCHLIST	*user_id, movie_id*, added_on	user_id → APP_USER, movie_id → MOVIE (cascade)
THEATRE	theatre_id, theatre_name, city, address	UNIQUE (theatre_name, city)
SCREEN	screen_id, *theatre_id*, screen_no, screen_type	theatre_id → THEATRE; UNIQUE (theatre_id, screen_no)
SEAT	seat_id, *screen_id*, seat_row, seat_number, seat_type	screen_id → SCREEN; UNIQUE (screen_id, seat_row, seat_number)
MOVIE_SHOW	show_id, *movie_id*, *screen_id*, show_start, base_price	movie_id → MOVIE, screen_id → SCREEN; UNIQUE (screen_id, show_start)
BOOKING	booking_id, *user_id*, *show_id*, booking_time, status	user_id → APP_USER, show_id → MOVIE_SHOW; UNIQUE (booking_id, show_id)
BOOKING_SEAT	*booking_id, seat_id*, *show_id*, price_paid	(booking_id, show_id) → BOOKING (cascade), seat_id → SEAT; UNIQUE (show_id, seat_id)
PAYMENT	payment_id, *booking_id*, amount, payment_method, payment_status, paid_at	booking_id → BOOKING; UNIQUE booking_id (makes it 1:1)

How we mapped things: SCREEN and SEAT technically have composite keys as weak entities (a seat's real key is theatre + screen_no + row + number). We gave them surrogate keys (screen_id, seat_id) so BOOKING_SEAT doesn't need a 4-column foreign key, but kept the natural keys as UNIQUE NOT NULL so they're still candidate keys. The seat specialization is mapped to a single table with seat_type as the discriminator, since the subclasses don't have any attributes of their own. Two extra tables would just mean extra joins for nothing.

Referential actions: we only used ON DELETE CASCADE on the pure link tables (genres, cast, reviews, watchlist), where a row means nothing once its parent is gone. Everything on the booking side uses the default (restrict), because you shouldn't be able to delete a user or a movie that has bookings and payments. That would wipe out financial history.

6. Important Integrity Constraints

Here are the main rules and what actually enforces each one. We tried to do as much as possible with plain constraints. The only integrity rule that needed a trigger is rule 10.

#	Rule	Enforced by
1	Username and email are unique for each user	UNIQUE constraints
2	A user can review a movie only once (they can edit it, not add a second one)	Composite PK (user_id, movie_id) on REVIEW
3	Rating is 0.5 to 5.0 in half-star steps	CHECK (rating BETWEEN 0.5 AND 5 AND MOD(rating*2,1)=0)
4	Certificate is U, UA, A or S; duration is 30-300 min	CHECK constraints on MOVIE
5	Screen numbers unique inside a theatre; seat labels unique inside a screen	UNIQUE (theatre_id, screen_no), UNIQUE (screen_id, seat_row, seat_number)
6	Two shows can't start on the same screen at the same time	UNIQUE (screen_id, show_start)
7	A booking must be for a real user and a real show	Foreign keys
8	**A seat can't be booked twice for the same show**	UNIQUE (show_id, seat_id) on BOOKING_SEAT
9	The show_id copied into BOOKING_SEAT must match the booking's show	Composite FK (booking_id, show_id) → BOOKING
10	A booked seat must be in the screen where the show is running	Trigger TRG_BOOKING_SEAT_SCREEN (raises ORA-20001)
11	Booking status is PENDING / CONFIRMED / CANCELLED	CHECK
12	One payment per booking; method is UPI / CARD / NETBANKING / WALLET	UNIQUE (booking_id) + CHECK
13	A SUCCESS or REFUNDED payment must have a paid_at time	Multi-column CHECK on PAYMENT
14	Prices, amounts and seat numbers must be positive	CHECK (> 0)

Why a trigger for rule 10? A CHECK constraint can only look at the row being inserted, and a foreign key can only check that a value exists, not that two looked-up values are equal (SEAT.screen_id = MOVIE_SHOW.screen_id). Our trigger only reads MOVIE_SHOW and SEAT, so there's no mutating-table problem. We tested every rule in this table

in 06_booking_integrity.sql, and each invalid insert fails with the expected Oracle error.

7. Normalization and Design Justification

Starting point. Our first idea was a single "ticket" table with everything on it: booking_id, username, email, movie title, genres, theatre, city, screen, show time, seats, price, status. It had all the classic problems. If a show's price changed you'd have to update every booking row (update anomaly). You couldn't add a show until someone booked it (insert anomaly). And deleting the only booking for a show would make you lose the show completely (delete anomaly).

- **1NF:** genres ("Action, Drama") and seats ("B1, B2") were lists, so they got their own rows/tables (MOVIE_GENRE, one row per seat).
- **2NF:** with key (booking_id, seat), almost everything depended only on booking_id, so we split it into BOOKING and BOOKING_SEAT. Only price_paid depends on the whole key.
- **3NF:** email depends on the user, the movie and price depend on the show, and city depends on the theatre. These are all transitive dependencies, so they moved to APP_USER, MOVIE_SHOW, SCREEN and THEATRE.

After this, every table is in BCNF (the only determinants are candidate keys) except one, which we kept in 3NF on purpose:

BOOKING_SEAT (booking_id, show_id, seat_id, price_paid) is 3NF but not BCNF. It has two candidate keys, {booking_id, seat_id} and {show_id, seat_id}, plus the FD booking_id → show_id. booking_id isn't a superkey, so it's not BCNF. But show_id is a prime attribute, so 3NF is fine with it. We accepted this because having show_id there is exactly what lets a simple UNIQUE (show_id, seat_id) stop double booking without any PL/SQL. The usual risk of the copy getting out of sync is handled by the composite foreign key (rule 9). Getting to BCNF here would have cost us that constraint, which is the textbook "BCNF isn't always dependency preserving" situation.

Other design decisions we made:

- **No stored totals.** SCREEN has no capacity column and BOOKING has no total_amount. Both can be calculated (COUNT of seats, SUM of price_paid), so storing them would just create update anomalies. Our views (e.g. BOOKING_SUMMARY, MOVIE_RATING_SUMMARY) calculate them instead.
- **price_paid looks redundant but isn't.** It records what the customer was actually charged at that moment. If the theatre changes the show price later, old bookings should keep their original price. The same goes for PAYMENT.amount (it could include a discount).
- **Rating and review in one table.** On Letterboxd a rating and a review are the same "log", so REVIEW has a nullable review_text. That allows rating-only entries without a second table with the exact same key.
- **Natural keys kept as UNIQUE.** We used surrogate IDs for joins, but all real-world keys (title+release date, theatre+city, screen+start time, etc.) are UNIQUE, so duplicates can't sneak in.

8. Indexes and PL/SQL Design

Indexes. Oracle automatically indexes every PRIMARY KEY and UNIQUE constraint, but not foreign keys. We added six indexes on the foreign keys the app filters on: BOOKING(user_id), BOOKING(show_id), MOVIE_SHOW(movie_id), REVIEW(movie_id), WATCHLIST(movie_id) and MOVIE_GENRE(genre_id).

Object	What it does / why
get_seat_price, get_avg_rating, seats_left (functions)	Premium = base + Rs 100 written in one place; average rating and seats left computed on the fly instead of stored.
book_seats (procedure)	Creates a PENDING booking and all its seats as one unit. If any seat fails, it rolls back to a savepoint, so it's all seats or none.
make_payment (procedure)	MERGE into PAYMENT (one row per booking) and sets the booking to CONFIRMED. SELECT … FOR UPDATE stops a booking being paid twice at once.
cancel_booking (procedure)	Frees the seats, keeps the booking row as CANCELLED and marks the payment REFUNDED. Refused once the show has started.
save_review (procedure)	MERGE = insert or update, so rule 2 (one review per user per movie) becomes "edit your review".
TRG_REVIEW_CLEARS_WATCHLIST (trigger)	When you review a movie, it's removed from your watchlist.

- **The procedures never COMMIT.** The caller decides, so booking + payment can be one transaction. The API commits only if every statement in the request worked, and rolls back otherwise.
- **No "is the seat free?" check before inserting.** Two users could both see a seat as free at the same moment. Instead we rely on UNIQUE (show_id, seat_id): Oracle checks it at insert time, so the second booking waits for the first and then fails.